Projet MLSF — Partie I : un solveur SAT pour le Meowdoku

Rapport — binôme : DAL MONTE Nicolò, DE MEDEIROS André Filipe

Octobre 2026

Introduction

Ce rapport décrit, question par question, les décisions de conception, les résultats obtenus et leur analyse. Tout le code est en Python (≥ 3.12) ; le solveur livré (`sat.py`) n'utilise que la bibliothèque standard. Le fichier `README.md` décrit l'arborescence et la façon de reproduire chaque résultat.

Jeux de données. La validation et les mesures s'appuient sur :

- les **instances publiques fournies par les enseignants** (`data/public-instances/`) : 41 fichiers DIMACS — 5 tests d'interface, 6 cas limites (formule vide, formule sans clause, clause vide, tautologie, littéraux répétés, littéraux tous négatifs) et 30 instances SATLIB variées (19 SAT, 11 UNSAT, 42 à 1 363 variables) — avec leur statut attendu (`expected.json`) ;
- des familles classiques de la bibliothèque **SATLIB** (Hoos & Stützle) : 3-SAT aléatoire au seuil (`uf20`, `uf50`, `uuf50`, `uf75`, `uuf75`, `uf100`, `uuf100`), `aim`, `dubois`, `pigeon-hole`, `flat50` (coloration de graphes) ; le statut SAT/UNSAT y est connu par construction ;
- des instances que nous générons (`experiments/gen_cnf.py`) : 3-SAT aléatoire à trois ratios clauses/variables, principe des tiroirs PHP(n+1, n), et surtout des grilles Meowdoku (à solution plantée ou à régions aléatoires). Leur statut attendu est calculé par un solveur de référence indépendant (`pycosat`, liaison Python de PicoSAT), uniquement pour la validation.

Usage des assistants IA. L'utilisation de l'intelligence artificielle (IA) au cours de ce projet a été réalisée avec modération et discernement. Des assistants IA ont été utilisés comme interlocuteur pour discuter et comparer des idées. Il est important de souligner que chaque recommandation générée par l'IA a été rigoureusement évaluée par les étudiants. Aucune suggestion n'a été acceptée aveuglément : ce n'est qu'après une validation humaine minutieuse que les solutions ont été dûment implémentées et testées.

Question 1 — Modélisation du Meowdoku

Variables

Pour une grille N × M, on introduit une variable $x_{i,j}$ par case, vraie ssi un chat occupe la case (i, j). Numérotation DIMACS : $\text{var}(i, j) = (i-1)\,M + j \in \{1, \ldots, NM\}$ (ordre ligne par ligne). Le décodage d'une valuation témoin v est immédiat : les chats sont sur les cases $\{(i, j) \mid v(x_{i,j}) = 1\}$.

Clauses

Notons $R_1, \ldots, R_K$ les régions.

Famille	Contrainte	Clauses	Nombre
R≥1	au moins un chat par région	$\bigvee_{c \in R_k} x_c$	K
R≤1	au plus un chat par région	$\neg x_c \vee \neg x_{c'}, c \neq c' \in R_k$	$\sum_k \binom{\lvert R_k \rvert}{2}$

Famille	Contrainte	Clauses	Nombre
L≤1	au plus un chat par ligne	$\neg x_{i,j} \vee \neg x_{i,j'}, j < j'$	$N\binom{M}{2}$
C≤1	au plus un chat par colonne	$\neg x_{i,j} \vee \neg x_{i',j}, i < i'$	$M\binom{N}{2}$
ADJ	pas de chats voisins en diagonale	$\neg x_{i,j} \vee \neg x_{i+1,j\pm1}$	$2(N-1)(M-1)$
L≥1 *(redondante)*	si $K = N$: un chat par ligne	$\bigvee_j x_{i,j}$	N
C≥1 *(redondante)*	si $K = M$: un chat par colonne	$\bigvee_i x_{i,j}$	M

Choix de conception.

- *Adjacence.* Le 8-voisinage comprend des voisins horizontaux, verticaux et diagonaux. Les deux premiers sont déjà interdits par L≤1 et C≤1 : seules les diagonales demandent des clauses spécifiques (deux par case, vers la ligne suivante, ce qui couvre chaque paire une seule fois).
- *Codage « par paires » des contraintes « au plus un ».* Il n'utilise que les variables $x_{i,j}$, comme demandé, et ne produit que des clauses binaires, idéales pour la propagation unitaire : poser un chat élimine immédiatement toutes les cases de sa ligne, de sa colonne, de sa région et ses voisines diagonales. Son défaut est un nombre quadratique de clauses ($O(N^3)$ pour N = M = K, davantage si une région est très grande). Nous avons aussi implémenté (option `--amo seq`) le **compteur séquentiel** de Sinz (2005), linéaire mais introduisant des variables auxiliaires ; avec le codage par paires, une grille 50 × 50 donne déjà 186 236 clauses, résolues en 1,1 s de CPU par le solveur optimisé : le codage par paires reste le choix par défaut.
- *Dédoublonnage.* Deux cases d'une même région **et** d'une même ligne engendreraient deux fois la même clause : l'encodeur ne l'émet qu'une fois (sur la grille de la figure 1 : 321 clauses au lieu de 420).
- *Clauses redondantes L≥1 / C≥1.* Lorsque $K = N$, il y a K chats, au plus un par ligne, sur N = K lignes : chaque ligne en contient exactement un (principe des tiroirs). Ces clauses ne changent donc pas l'ensemble des modèles mais rendent accessible à la simple propagation unitaire une déduction qui demanderait sinon un raisonnement par l'absurde coûteux (« la seule case encore possible de la ligne doit porter un chat »). Leur effet est mesuré en Q5.
- *Cas général.* L'encodage traite N, M, K quelconques ; si $K > \min(N, M)$ la formule est (correctement) insatisfiable.

Correction. Une valuation satisfait R≥1 ∧ R≤1 ssi chaque région contient exactement un chat ; L≤1, C≤1 expriment directement les règles 1 et 2 ; L≤1 ∧ C≤1 ∧ ADJ interdisent toute paire de chats 8-voisins. Les modèles de la formule (restreints aux $x_{i,j}$) sont donc exactement les solutions de la grille.

Fichiers. `meowdoku/grid.py` lit/écrit le format de la figure 1b (lignes `c` ignorées — sauf cas limite d'une région nommée `c`, géré —, `GRID`, `ZONES`, contrôles : dimensions, régions déclarées, régions non vides). `cli/meow2cnf.py grille.txt -o grille.cnf` produit le DIMACS, avec en commentaire la table des variables et le décompte des clauses par famille.

Générateur de grilles

`cli/generate_grids.py N [--M M --K K] [--count c] [--unique] [--random]` (`meowdoku/generate.py`). Méthode **par solution plantée** :

1. *Placement des K chats* : recherche aléatoire avec retour arrière (lignes distinctes tirées au hasard, colonnes mélangées) d'un ensemble de K cases sur des lignes et colonnes distinctes et non adjacentes. Pour N = M = K, c'est une permutation σ avec $|\sigma(i) - \sigma(i+1)| \geq 2$ (il en existe pour N = 1 et N ≥ 4 ; N = 2, 3 sont impossibles).

2. *Croissance des régions* : chaque chat est le germe d'une région ; on tire au hasard une case de la frontière commune et on l'attribue à la région qui l'a proposée, jusqu'à couvrir la grille. Les régions sont connexes, forment une partition et chacune contient exactement son chat : **le placement de l'étape 1 est une solution par construction.** Les noms de régions sont affectés après mélange, pour qu'ils ne trahissent pas la position du chat.
3. *Unicité (option `--unique`)* : une vraie grille de jeu a une solution unique. On cherche avec notre solveur une solution $S' \neq S$ (en bloquant S par la clause $\bigvee_{c \in S} \neg x_c$). Si elle existe, on choisit une case $c \in S' \setminus S$ et on la cède à une région voisine, à condition que sa région d'origine reste connexe. S' devient illicite (sa région d'origine n'a plus de chat, la région d'accueil en a deux) alors que S reste licite (c n'est pas un germe). On itère jusqu'à unicité. En pratique : quelques secondes jusqu'à 12×12 ; au-delà, le nombre de solutions concurrentes rend la méthode lente (limite assumée de l'option).

L'option `--random` tire au contraire des germes quelconques : la grille n'a alors **aucune garantie de solution** ; elle sert à produire des instances UNSAT réalistes pour la question 5 (25 de nos 40 grilles aléatoires le sont).

Question 2 — Lecture DIMACS et algorithmes 1 et 2

Structure de données (`solver/cnf.py`)

Un littéral est un entier non nul (convention DIMACS : `k` pour x_k, `-k` pour $\neg x_k$) : négation et variable sont en O(1) (`-l`, `abs(l)`). Une formule est un objet `CNF(num_vars, clauses)`. Le lecteur tolère les particularités des fichiers réels et vérifie la présence de l'en-tête ; un mode `strict` contrôle la cohérence avec l'en-tête :

- une clause est lue **jusqu'à son `0`**, même répartie sur plusieurs lignes : 6 des instances publiques (`public-04`, `07`, `12`, `14`, `21`, `23`) placent le `0` final sur la ligne suivante ;
- ligne `%` de fin des fichiers SATLIB, espaces multiples, clause vide (ligne réduite à `0`), formule sans clause (`p cnf 3 0`, `p cnf 0 0`) ;
- à l'inverse, `dubois100.cnf` (SATLIB) omet le `0` final de nombreuses lignes : lire « jusqu'au 0 » fusionnerait des clauses (598 au lieu de 800) et rendrait la formule satisfiable. Lorsque le nombre de clauses lues ne correspond pas à l'en-tête mais que la lecture « une ligne = une clause » (règle de l'énoncé) y correspond, c'est cette dernière qui est retenue.

Pour DPLL, chaque clause devient un `frozenset` : appartenance en O(1), élimination des doublons, et surtout possibilité de **partager** entre père et fils les clauses non modifiées par une affectation. Les clauses tautologiques ($p \vee \neg p$) sont retirées à la lecture (toujours vraies, elles fausseraient la règle du littéral pur).

Algorithmes (`solver/dpll.py`)

Implémentation fidèle au pseudo-code : récursive, sans effet de bord.

- `assign(F, l)` calcule $F[l \leftarrow 1]$: on supprime les clauses contenant l et on retire $\neg l$ des autres ; $F[p \leftarrow 0]$ s'écrit `assign(F, -p)`.
- Une **seule passe** (`scan`) sur F traite les lignes 1 à 11 : détection de la clause vide, d'une clause unitaire, et calcul de l'ensemble des littéraux (un littéral l est pur ssi $\neg l$ n'y figure pas).
- `dpll1` renvoie SAT/UNSAT ; `dpll2` renvoie en plus une valuation (un `dict`, enrichi en remontant la récursion : « $v \cup \{l\}$ »). Les variables disparues de la formule sans avoir été affectées peuvent prendre n'importe quelle valeur : on les complète à `False`.
- `RandomChoice` : variable uniforme parmi celles de F, valeur uniforme ; le générateur pseudo-aléatoire est **graine-able** (reproductibilité).

- La profondeur de récursion peut atteindre le nombre de variables : l'appel est exécuté dans un fil d'exécution à grosse pile avec une limite de récursion relevée (`run_deep`).

Complexité. Chaque nœud coûte $O(|F|)$ (une passe d'analyse + la construction de $F[l \leftarrow b]$), et la mémoire est en $O(\text{profondeur} \times |F|)$ dans le pire cas (en pratique bien moins grâce au partage des clauses intactes). Le nombre de nœuds est exponentiel dans le pire cas (2^n).

Validation (`experiments/validate.py`)

180 instances (110 SAT / 70 UNSAT) : les **41 instances publiques des enseignants** (statut lu dans `expected.json`) et 139 instances SATLIB et générées. Chaque instance est résolue par 19 solveurs (algorithmes 1 et 2, DPLL_H avec chacune des 16 configurations d'heuristiques, CDCL), avec une limite de 20 s (`results/validation/`) : **3 420 exécutions, aucune réponse fausse, tous les modèles renvoyés vérifiés** (chaque clause contient un littéral vrai) et aucune contradiction de statut entre solveurs.

Sur les 41 instances publiques, les 6 cas limites et les 5 tests d'interface sont traités correctement par tous les solveurs. Les DPLL résolvent 35 à 40 instances sur 41 en 20 s (algorithmes 1 et 2 : 36 ; H8 : 40), le CDCL les 41. `public-03`, identique clause pour clause à `dubois21` de SATLIB (63 variables, UNSAT), résiste à tous les DPLL, comme toute la famille `dubois` (Q5). Les autres exécutions non terminées concernent `aim-100`, `dubois`, `hole9`, deux grilles 20 × 20 et deux grilles à régions aléatoires.

Les tests unitaires (`tests/`, 71 tests) comparent en outre tous les solveurs à une énumération exhaustive sur 40 petites formules aléatoires (dont formules vides, clause vide, clauses unitaires, tautologies), vérifient les exemples de l'énoncé, et exécutent `sat.py` dans les deux modes sur les 11 tests d'interface et cas limites publics.

Performances

Les mesures détaillées sont en Q5. En résumé pour l'algorithme 2 (choix aléatoire), en temps CPU (moyenne géométrique) : 1,8 ms pour `uf20`, 38 ms pour `uf50`, 0,23 s pour `uuf50`, 1,3 s pour `uf75`, 5,0 s pour `uuf75`, et échec (> 20 s) sur `dubois20` (60 variables seulement !). La croissance est clairement exponentielle en n, et **les instances UNSAT sont nettement plus coûteuses** que les SAT de même taille : il faut parcourir tout l'arbre, alors qu'une instance SAT s'arrête à la première feuille satisfaisante. Le coût par nœud est dominé par la recopie de la formule. L'algorithme 1 a le même arbre de recherche que l'algorithme 2 (même graine) ; il économise seulement la construction de la valuation, négligeable : sur les 103 instances de la validation résolues par les deux en plus de 10 ms, son temps vaut 0,99 fois celui de l'algorithme 2 (moyenne géométrique), avec exactement les mêmes délais dépassés.

Question 3 — Résolution d'une grille

`cli/solve_meowdoku.py grille.txt -o solution.txt` enchaîne : grille → CNF → **écriture d'un fichier DIMACS** → relecture de ce fichier et résolution par notre solveur → décodage → **vérification indépendante** des quatre règles (`check_solution`, qui ne réutilise pas l'encodage CNF) → fichier solution au format de la figure 2 (recopie de l'entrée, dictionnaire région ↦ (ligne, colonne), grille de *).

Le solveur se choisit par option : `--solver dpll` (algorithme 2), `--solver dpllh` (DPLL_H, par défaut avec H9, meilleure configuration DPLL en Q5) ou `--solver cdcl` (défaut). Le CDCL est le cœur du mode `optimized` de la Q6 ; il résout une grille 50 × 50 en 1,1 s de CPU, alors qu'en 24 × 24 les DPLL demandent déjà 9 à 40 s avec les heuristiques de l'énoncé (PAR-2 géométrique, échecs compris ; 2,1 s avec H9, Q5).

Sur la grille de la figure 1, les trois solveurs produisent une sortie identique à la figure 2 (`data/examples/grille1.sol.txt`), et la recherche d'une seconde solution (méthode du générateur, ci-dessus)

confirme que cette solution est unique. `cli/verify_solution.py` vérifie un fichier solution existant (cohérence dictionnaire/grille de * et règles du jeu). Si la grille est illicite (formule UNSAT), le programme le signale et renvoie un code d'erreur.

Question 4 — Heuristiques de branchement

Implémentation (`solver/heuristics.py`, `solver/dpll_h.py`)

`DPLLH` reprend exactement l'algorithme 2, `RandomChoice` étant remplacé par `heuristic.choose(F, lits)`. Une heuristique expose `prepare(F₀)` (appelée une fois sur la formule initiale) et `choose(F, lits)` (à chaque décision).

- « Variable non encore affectée » = variable présente dans la formule courante (une variable absente est affectée ou sans influence : $F[p \leftarrow b] = F$, la choisir serait une décision inutile).
- **Versions statiques** : `prepare` calcule une fois pour toutes un ordre de couples (variable, valeur) sur F_0 ; une décision prend le premier couple dont la variable est encore présente.
- **Versions dynamiques** : les compteurs sont recalculés sur la formule courante à chaque décision. Le comptage utilise `collections.Counter.update` (boucle en C) ; les poids de Jeroslow-Wang sont calculés en regroupant les clauses par taille (un `Counter` par taille puis une combinaison pondérée), ce qui évite une boucle Python par littéral.
- Départage des ex æquo : plus petit indice (toutes les heuristiques non aléatoires sont déterministes, donc les mesures reproductibles).
- **Complexité** (n variables, $|F|$ = nombre total de littéraux) : une version statique coûte $O(|F_0| + n\log n)$ une seule fois (comptage puis tri) et $O(n)$ au pire par décision (parcours de l'ordre jusqu'à la première variable encore présente) ; une version dynamique coûte $O(|F|)$ par décision (comptage complet de la formule courante), soit le même ordre que la construction de $F[l \leftarrow b]$: c'est ce qui explique, en Q5, que la part de l'heuristique passe d'environ 10 % à près de la moitié du temps.

Nous avons implémenté **toutes** les heuristiques de l'énoncé, dans leurs deux variantes : H0 (CA), H1 (PVL), H2–H7 en `s` et `d` (VPPM, LP, VFCM/MOMS, LFCM, VJW, LJW), soit 14 configurations, auxquelles s'ajoutent deux heuristiques personnelles. Les contraintes de l'énoncé sont donc couvertes (sélection de littéral : H3, H5, H7 ; taille des clauses : H4–H7 ; dynamiques : H2d–H7d).

Adaptations (signalées). (1) Pour H4s/H5s, la taille minimale k est calculée sur les clauses *non unitaires* de F_0 : les clauses unitaires initiales sont propagées avant toute décision, et k = 1 rendrait l'ordre statique dénué de sens. (2) Pour H4s/H5s, de nombreuses variables ont $\#_k = 0$: on départage alors par $\#(p, F_0)$ plutôt que par indice. (3) Dans l'énoncé de H5, « 1 si l est de la forme ¬p » est une coquille : nous affectons 0 dans ce cas (ce que confirme la phrase suivante de l'énoncé).

Conformité. Les heuristiques sont validées comme l'algorithme 2 (Q2) : mêmes statuts, modèles vérifiés, et test unitaire reproduisant l'exemple de l'énoncé ($\sum_k \#_k(x_2, F)\, 2^{-k} = 7/16$, H6 choisit $x_2 \leftarrow 1$).

Heuristiques personnelles

- **H8 — MOMS-produit** (inspirée de POSIT, Freeman 1995). Sur les clauses de taille minimale : $\text{score}(p) = \text{occ}_k(p)\,\text{occ}_k(\neg p)\,2^{10} + \text{occ}_k(p) + \text{occ}_k(\neg p)$. Le produit favorise les variables présentes avec **les deux** polarités : chacune des deux branches raccourcit alors beaucoup de clauses minimales, donc produit des propagations unitaires des deux côtés — c'est précisément ce qui réduit l'arbre lorsqu'il faut explorer les deux branches (instances UNSAT).

- **H9 — MRV, « plus petite clause positive ».** Conçue après les premières mesures de Q5 (voir ci-dessous) : on choisit la plus petite clause dont tous les littéraux sont positifs, et on y affecte à 1 la variable la moins

contraignante (celle dont le littéral négatif apparaît le moins dans les clauses binaires). Sur le Meowdoku, les clauses positives sont exactement les contraintes « au moins un chat » (région, ligne, colonne) : on traite d'abord la région/ligne/colonne ayant le moins de cases possibles (heuristique *Minimum Remaining Values* des solveurs de contraintes) et on y pose un chat. Repli sur H6d s'il n'existe aucune clause entièrement positive. Le choix de la case dans la clause a été mesuré (`experiments/h9_variants.py`, `results/benchmark/h9_variants.*`) sur les 27 grilles les plus difficiles pour DPLL (plantées 16 × 16 à 24 × 24, régions aléatoires 12 × 12 à 16 × 16), limite de 30 s de CPU :

case choisie	résolues	PAR-2 moyen
la moins contraignante (retenue)	**27 / 27**	**1,2 s**
plus petit indice	27 / 27	2,1 s
la plus contraignante	25 / 27	5,5 s

Poser le chat qui élimine le moins de cases laisse le plus de possibilités aux autres régions : on s'engage le moins possible, ce qui limite les retours arrière.

Pourquoi ces choix. Les heuristiques de l'énoncé couvrent deux axes indépendants (variable vs littéral ; fréquence brute vs clauses courtes vs pondération JW) croisés avec statique/dynamique : les implémenter toutes coûte peu (une trentaine de lignes chacune, grâce à des fonctions de comptage partagées) et permet en Q5 une comparaison *toutes choses égales par ailleurs*. Les deux heuristiques personnelles testent deux idées : favoriser les propagations dans les deux branches (H8, ciblant l'UNSAT) et exploiter la structure « exactement un » du Meowdoku (H9).

Question 5 — Instrumentation et analyse des performances

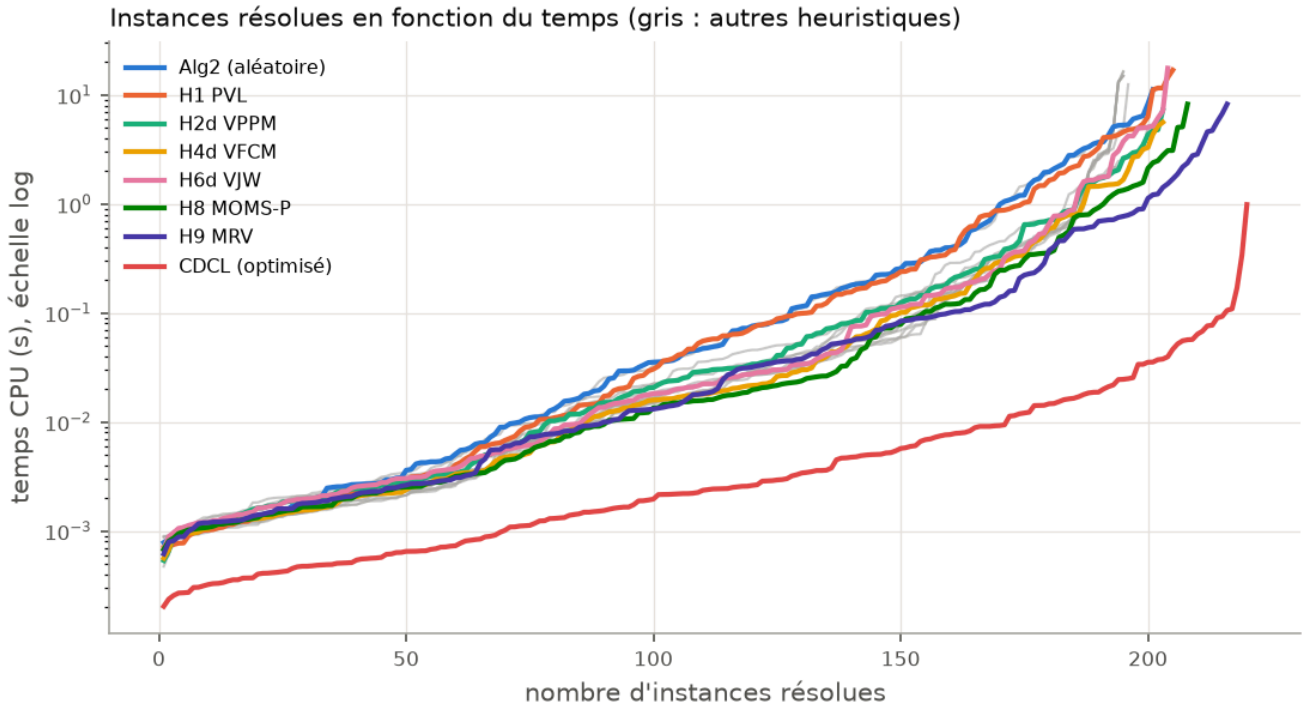
Protocole

- **Instrumentation** (`solver/dpll_h.py`, classe `Stats`) : les compteurs demandés sont incrémentés aux lignes correspondantes de l'algorithme 3 — décisions (ligne 17), retours arrière (branche `else`, ligne 21), propagations unitaires (ligne 5), littéraux purs (ligne 11) — ainsi que nombre de nœuds, de conflits (clause vide), profondeur maximale, temps total et temps passé dans l'heuristique (`prepare` + `choose`), d'où le **ratio de temps consacré à l'heuristique**.
- **Instances** (`experiments/benchmark.py`) : 220 instances en 3 catégories — *3-SAT aléatoire* (`uf20`, `uf50`, `uuf50`, `uf75`, `uuf75` et nos `rand3` à trois ratios clauses/variables), *structurées* (`aim-50`, `dubois20–23`, PHP, `flat50`) et *Meowdoku* (`meow_sat` : grilles plantées N = 5..24 ; `meow_nored` : les mêmes sans clauses redondantes ; `meow_rnd` : grilles à régions aléatoires, SAT ou UNSAT). 145 instances SAT, 75 UNSAT.
- **Solveurs** : algorithme 2, les 16 configurations de DPLL_H et le solveur optimisé (CDCL, Q6) comme référence. Les solveurs aléatoires (Alg2, H0) sont lancés avec 3 graines : **4 840 exécutions**.
- **Mesure** : chaque exécution a lieu dans un processus fils, limite de 20 s de **temps CPU** (`time.process_time`, insensible à la charge des autres processus). Le score **PAR-2** compte une exécution non terminée pour 2 × la limite (40 s) ; la colonne « PAR-2 moyen » est une moyenne arithmétique, les compteurs et les rapports sont des moyennes **géométriques** (les valeurs s'étalent sur 5 ordres de grandeur). Machine : Linux (WSL2), Intel Core i5-1235U, Python 3.12.3, 6 processus en parallèle (un par cœur physique) ; campagne complète : 28 min. Les compteurs sont exacts et indépendants de la machine : une première campagne sous Windows a donné exactement les mêmes. (Sous Windows, `time.process_time` n'avance que par pas de 15,6 ms ; `solver.dpll.clock` y bascule sur `time.perf_counter`.)
- **Correction** : 0 réponse fausse sur les 4 840 exécutions, tous les modèles vérifiés.

Tableaux complets : `results/benchmark/tables.md` ; figures : `results/benchmark/fig_*.png`. Reproduction : `python experiments/benchmark.py --timeout 20 --jobs 6 && python experiments/analyze.py`.

Vue d’ensemble

solveur	résolues	SAT	UNSAT	PAR-2 moyen	heur. %
Alg2 / H0 (aléatoire)	90,8 %	93,6 %	85,3 %	4,28 s	3
H1 PVL	93,2 %	94,5 %	90,7 %	3,48 s	3
H2s–H7s (statiques)	88,6–89,1 %	90–91 %	85,3 %	4,6–4,8 s	9–14
H2d–H7d (dynamiques)	92,3–92,7 %	96–97 %	85,3 %	3,3–3,4 s	37–47
H8 MOMS-produit	94,5 %	97,2 %	89,3 %	2,46 s	34
H9 MRV	**98,2 %**	**100 %**	**94,7 %**	**1,06 s**	16
CDCL (Q6)	100 %	100 %	100 %	0,016 s	—



Nombre d’instances résolues en fonction du temps (graine 0 pour les solveurs aléatoires).

Le classement global masque des comportements très différents selon la famille ; nous les analysons séparément ci-dessous (compteurs : moyennes géométriques sur les 190 instances résolues par tous les solveurs DPLL).

L’heuristique améliore-t-elle l’algorithme 2 ?

Sur le 3-SAT aléatoire et les instances structurées : oui, nettement.

décisions (moy. géo.)	3-SAT SAT	3-SAT UNSAT	structuré SAT	structuré UNSAT
H0 (aléatoire)	32	173	559	4 150
H1 PVL	26	159	326	3 100
H2s VPPM	12	37	59	1 770
H4d VFCM	9,3	18	37	726
H6d VJW	8,7	21	23	449
H8 MOMS-P	8,8	**16**	23	877

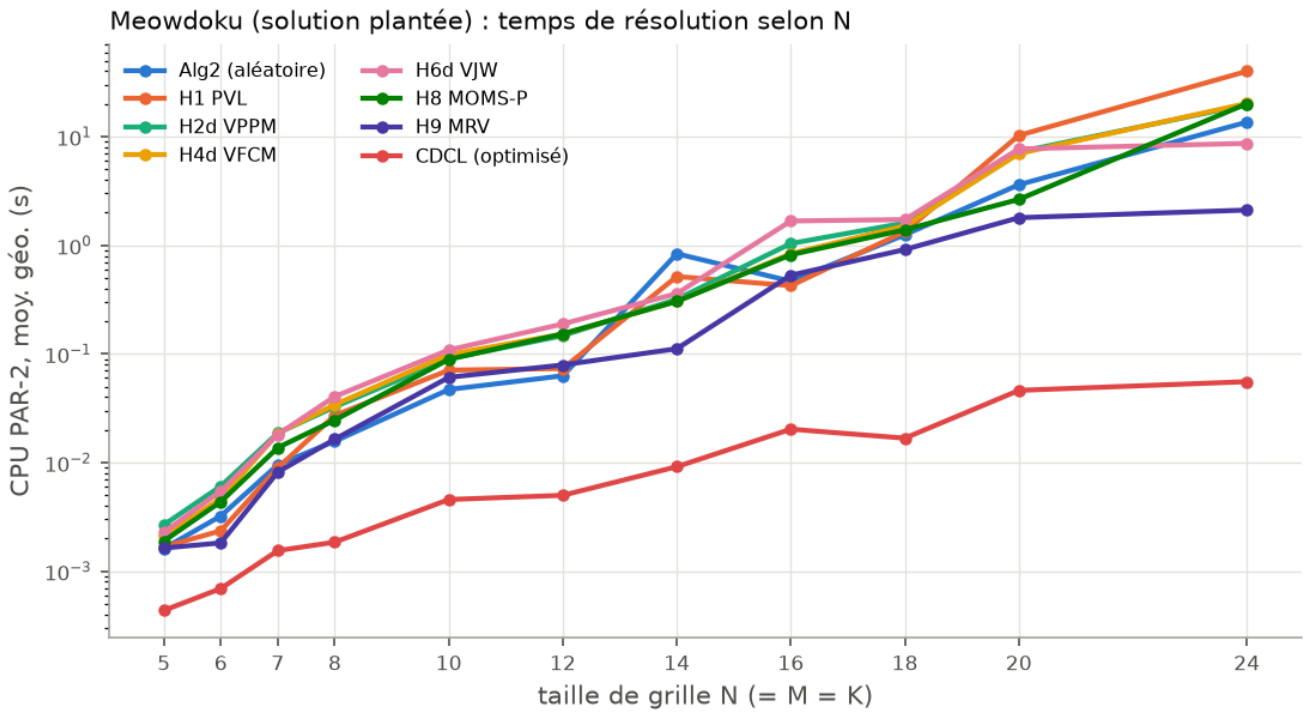
décisions (moy. géo.)	3-SAT SAT	3-SAT UNSAT	structuré SAT	structuré UNSAT
H9 MRV	20	64	**15**	**182**

Toute heuristique fondée sur les occurrences divise le nombre de décisions par 5 à 10 sur le 3-SAT UNSAT, et le temps en proportion : sur `uuf75` (75 variables, UNSAT), le PAR-2 passe de 5,0 s (Alg2) à 0,12 s (H8 ou H4d), soit un facteur 40 ; sur `aim`, de 1,6 s à 0,05 s (H6d), voire 0,014 s (H9). H1 (première variable libre) n'apporte presque rien : l'ordre DIMACS n'a aucun sens sur une instance aléatoire. **La famille `dubois` reste hors de portée de tous les DPLL** (60 à 69 variables seulement !) alors que CDCL la résout en 8 ms : ces formules (chaînes de XOR) imposent à DPLL de ré-explorer des sous-arbres d'échec identiques, ce qu'aucun ordre de branchement n'évite — c'est l'argument principal du solveur optimisé de la Q6.

Sur le Meowdoku : guère, sauf pour H8 et H9. Les heuristiques à polarité majoritaire de l'énoncé (H2–H7) font **plus de décisions que le hasard** (52 à 70 contre 20 sur les grilles SAT) et sont plus lentes (PAR-2 géométrique sur `meow_sat` : 0,11–0,22 s contre 0,12 s pour Alg2). Sur les grandes grilles plantées (N ≥ 16), les statiques résolvent *moins* que le hasard (50–58 % contre 72 %), les dynamiques à peine plus (75 %, sauf H6d : 92 %) ; sur les grilles à régions aléatoires, 73–77 % contre 73 %. À l'inverse, H1 — qui affecte toujours la valeur 1 — fait moins de décisions que le hasard (15 contre 20) et résout 90 % des grilles aléatoires : un premier indice de l'explication ci-dessous.

Meowdoku	décisions SAT	retours arr. SAT	décisions UNSAT
H0 (aléatoire)	20	6,4	8,9
H1 PVL (valeur 1)	15	6,8	8,1
H2s–H7s	52	7,2–7,5	19–20
H2d–H7d	56–70	0,7–1,4	17–21
H8 MOMS-P	42	1,0	2,3
H9 MRV	**8,1**	1,2	**1,4**

Explication : une variable $x_{i,j}$ apparaît **positivement** dans au plus 3 clauses (région, ligne, colonne) mais **négativement** dans O(N) clauses binaires « au plus un » et d'adjacence. Toutes les heuristiques à polarité majoritaire choisissent donc $x_{i,j} = 0$ (« pas de chat ici ») : une décision qui ne propage presque rien. Il faut alors 50 à 70 décisions là où le hasard, qui pose un chat une fois sur deux, en fait 20. Une décision $x_{i,j} = 1$ élimine au contraire d'un coup ligne, colonne, région et voisinage, par propagation unitaire. C'est cette observation qui a conduit à **H9** (Q4) : brancher sur la plus petite clause positive (la région, la ligne ou la colonne ayant le moins de cases encore possibles) avec la valeur 1. H9 résout **100 %** des grilles, y compris en 24 × 24, avec 2,5 fois moins de décisions que le hasard. H8, dont le score produit $\mathrm{occ}_k(p) \cdot \mathrm{occ}_k(\neg p)$ favorise les variables présentes dans les deux polarités parmi les clauses minimales, est également très efficace sur les grilles UNSAT (2,3 décisions).



Grilles plantées : temps de résolution selon N (PAR-2, moyenne géométrique).

La croissance est exponentielle en N pour tous les DPLL (pente quasi constante en échelle log) ; H9 garde une pente plus faible (2,1 s en 24 × 24 contre 9 à 40 s, avec des échecs, pour les autres), CDCL reste sous 0,1 s.

Statique ou dynamique ?

heuristique	décisions d/s	CPU d/s	heur. % s	heur. % d	résolues s / d
H2 VPPM	0,96	1,34	9	40	195 / 203
H3 LP	0,99	1,37	9	40	195 / 203
H4 VFCM	**0,75**	**0,99**	14	35	195 / 203
H5 LFCM	0,87	1,07	14	35	195 / 203
H6 VJW	**0,75**	1,21	10	45	196 / 204
H7 LJW	0,85	1,29	11	46	195 / 203

(rapports géométriques dynamique / statique sur les instances résolues par les deux)

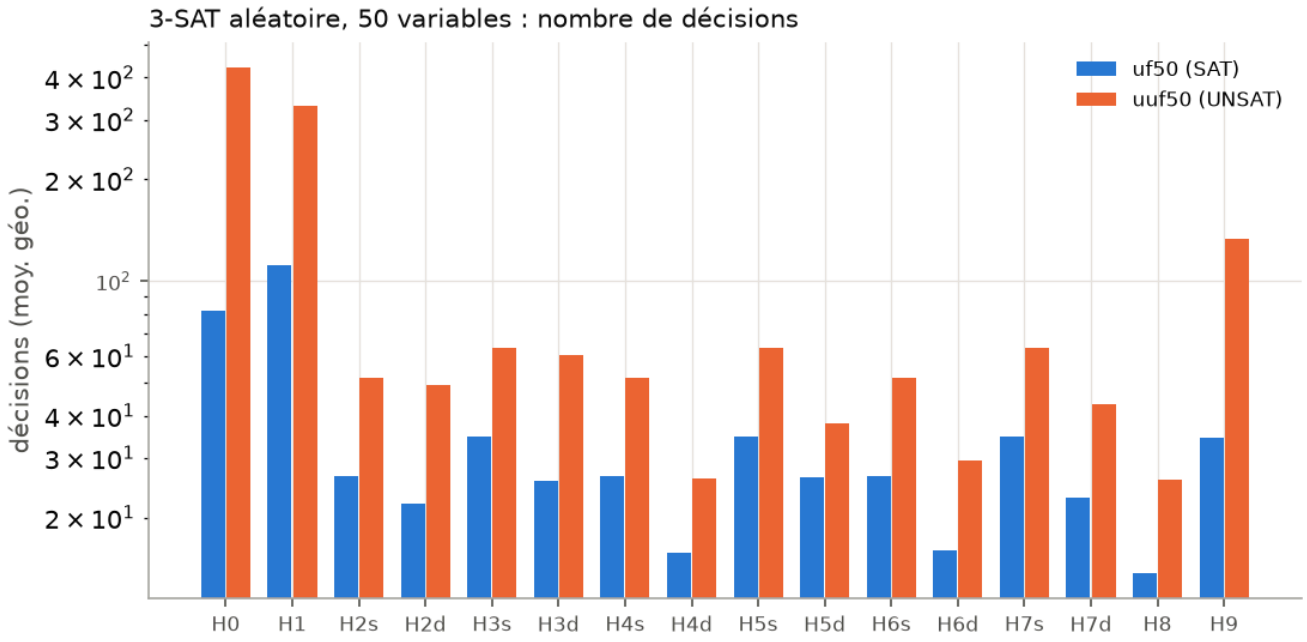
- Le recalcul **réduit l'arbre**, mais surtout pour les heuristiques qui dépendent de la **taille des clauses** (H4, H6 : −25 % de décisions). La fréquence brute d'une variable (H2, H3) évolue peu au cours de la recherche ; la taille des clauses, si : au départ toutes les clauses d'un 3-SAT ont la taille 3, de sorte que H4s et H6s produisent **exactement le même ordre** que H2s (compteurs identiques dans les tableaux), alors que H4d/H6d exploitent les clauses binaires créées par la recherche.
- Ce gain est **absorbé par le coût** : un recalcul en O(|F|) à chaque décision fait passer la part de l'heuristique de 9–14 % à 35–46 % du temps. Au total, le dynamique n'est rentable en temps que pour H4 (rapport 0,99), et coûte 7 à 37 % de plus pour les autres sur les instances faciles.
- En revanche le dynamique **résout plus d'instances** (203–204 contre 195–196) : la réduction de l'arbre est multiplicative (elle croît avec la taille de l'instance) alors que le surcoût est un facteur constant par nœud. Sur les instances difficiles, c'est le dynamique qui l'emporte (PAR-2 moyen 3,3–3,4 s contre 4,6–4,8 s).

Variable ou littéral ?

À critère égal, les heuristiques *variable + polarité majoritaire* (H2, H4, H6) battent leur version *littéral* (H3, H5, H7), surtout en UNSAT : 3-SAT UNSAT, 37 contre 48 décisions (statiques), 18 contre 26 (H4d / H5d), 21 contre 30 (H6d / H7d). Sur une instance UNSAT les deux branches sont explorées : ce qui compte est la réduction obtenue dans *les deux* branches, que mesure le nombre total d'occurrences $\#(p, F)$, et non celle d'une seule polarité.

SAT et UNSAT

- Les instances UNSAT demandent 5 à 10 fois plus de décisions que les SAT de même taille (3-SAT : 173 contre 32 pour H0) : il faut épuiser l'arbre, alors qu'une instance SAT s'arrête à la première feuille satisfaisante. Une bonne heuristique gagne donc davantage sur l'UNSAT (où elle doit provoquer des conflits tôt) que sur le SAT (où elle doit deviner la bonne polarité).
- Sur les instances UNSAT, retours arrière = décisions dans tous les tableaux : chaque décision échoue dans ses deux branches. C'est un contrôle de cohérence de l'instrumentation.
- `fig_decisions_uf50.png` illustre l'écart SAT / UNSAT pour chaque heuristique sur les instances à 50 variables.



3-SAT aléatoire à 50 variables : décisions par heuristique, SAT et UNSAT.

Autres observations

- **Littéraux purs.** Fréquents sur le 3-SAT et les instances structurées, **jamais** appliqués sur le Meowdoku : une variable ne perd ses occurrences positives que si sa clause « au moins un » est satisfaite, c'est-à-dire si un chat est posé dans sa région, et la propagation des clauses « au plus un » l'a alors déjà mise à 0. Son coût (calcul de l'ensemble des littéraux) y est donc pure perte.
- **Clauses redondantes L≥1 / C≥1 (Q1).** Sans elles, CDCL fait 3,8 fois plus de décisions (temps × 1,8), les heuristiques statiques 1,5 fois plus ; les dynamiques et H9, qui trouvent seules les « dernières cases libres », y sont presque insensibles (1,0–1,2).
- **Principe des tiroirs : l'heuristique ne compte plus.** Sur `hole9` (10 pigeons, 9 trous, UNSAT), toutes les configurations DPLL_H font **exactement 362 879 = 9! − 1 décisions** : la formule est invariante par permutation des pigeons et des trous, toutes les variables se valent et aucun ordre ne raccourcit l'arbre. Seul compte alors le coût par nœud : H1, dont le choix est presque gratuit, conclut en 22 s, contre 31 à 39 s

pour H2d–H9 et 52 s pour l'algorithme 2 (mesure complémentaire, `results/benchmark/pigeon_hole.csv`, limite 120 s de CPU ; commande dans le `README.md`). Le CDCL, lui, ne conclut pas en 120 s : l'apprentissage n'aide pas sur cette famille (toute preuve par résolution y est exponentielle) et son coût par conflit, élevé en Python, devient un handicap. Aucun solveur ne résout `hole10` en 120 s. Cette observation justifie le portfolio de la Q6.
- **Ratio heuristique.** Sur le Meowdoku SAT, les heuristiques dynamiques passent 65–70 % du temps à calculer leur score : les grilles ont beaucoup de clauses ($O(N^3)$) et peu de décisions utiles, ce qui renforce l'intérêt d'une heuristique bon marché et adaptée (H9 : 19 %).

Bilan : attentes et observations

- *Conforme aux attentes* : les heuristiques à base d'occurrences écrasent le hasard sur le 3-SAT (facteur 5 à 40) ; les heuristiques de type MOMS / Jeroslow-Wang sont les meilleures sur le 3-SAT UNSAT (H8, H4d, H6d) ; l'UNSAT est beaucoup plus coûteux que le SAT.
- *Moins que prévu* : le recalcul dynamique, attendu comme nettement supérieur, ne réduit l'arbre que de 0 à 25 % et coûte presque autant qu'il rapporte ; son intérêt n'apparaît que sur les instances difficiles.
- *Contraire à nos attentes* : sur le Meowdoku, les heuristiques à polarité majoritaire de l'énoncé (H2–H7) font plus de décisions que le hasard et ne résolvent guère plus de grilles, à cause de la polarité choisie (0). L'analyse des compteurs (beaucoup de décisions, peu de propagations) a directement guidé la conception de H9, la meilleure configuration DPLL.
- *Limite commune* : aucun ordre de branchement ne sauve DPLL sur `dubois` ; il faut mémoriser les conflits, d'où le choix d'un CDCL en Q6.
- *Contre-exemple* : sur le principe des tiroirs, c'est l'inverse — le DPLL le plus simple (H1) bat le CDCL, d'où le portfolio CDCL + DPLL de la Q6.

Question 6 — Solveur optimisé et interface imposée

Interface (`sat.py`)

```
python sat.py instance.cnf [--mode dpll|optimized] [--output-model]
```

- `--mode dpll` : algorithme 2 (Q2), choix aléatoire avec graine fixe (`--seed`, 0 par défaut) ;
- `--mode optimized` (défaut) : portfolio CDCL + DPLL décrit ci-dessous ;
- sortie standard réservée au résultat : `s SATISFIABLE` / `s UNSATISFIABLE` et, avec `--output-model` et si SAT, `v l1 l2 … 0` (toutes les variables de l'en-tête, positives = vraies ; `v 0` pour une formule sans variable) ; diagnostics éventuels sur stderr (`--verify` vérifie le modèle et l'annonce sur stderr) ;
- code de retour 0 (nous évitons la convention 10/20 des solveurs SAT, qu'une infrastructure pourrait interpréter comme une erreur) ;
- aucune dépendance hors bibliothèque standard ; toutes les mesures ci-dessous ont été faites dans l'image Docker `python:3.12-slim` (Python 3.12.15) avec les limites de la compétition (4 CPU, 8 Gio, 60 s).

Choix du solveur optimisé (`solver/cdcl.py`)

Conclusions de Q5 qui guident ces choix : (i) le coût par nœud de DPLL est dominé par la **recopie de la formule** ; (ii) même avec la meilleure heuristique, DPLL **ré-explore des sous-arbres d'échec identiques** (familles `dubois`, grilles aléatoires UNSAT) ; (iii) une bonne heuristique dynamique améliore l'arbre mais son coût de calcul, en $O(|F|)$ par décision, absorbe une grande part du gain. Le solveur optimisé garde le schéma DPLL (propagation, décision, retour arrière) mais remplace chaque composant :

Composant	DPLL (Q2/Q4)	Solveur optimisé
Formule	recopiée à chaque nœud	modifiée en place, affectations dans un tableau, pile d'affectations (*trail*) et niveaux de décision ; retour arrière = dépiler
Propagation unitaire	recherche d'une clause unitaire par parcours complet	**2 littéraux surveillés** : seules les clauses surveillant un littéral devenu faux sont visitées ; liste dédiée pour les **clauses binaires** (majoritaires dans le Meowdoku)
Littéral pur	appliqué	abandonné (coûteux à maintenir, peu utile avec l'apprentissage)
Conflit	retour chronologique	**analyse 1-UIP**, clause apprise minimisée, **retour non chronologique**
Heuristique	statique ou dynamique en $O(\|F\|)$	**VSIDS** : activité des variables augmentée lors des conflits et décroissance exponentielle — version dynamique à coût quasi nul de « variable la plus fréquente dans les conflits récents » ; tas binaire (`heapq`) à suppression paresseuse
Polarité	selon l'heuristique	**sauvegarde de phase**, polarité initiale **négative** (dans le Meowdoku presque toutes les $x_{i,j}$ sont fausses ; cf. l'observation de Q5 sur la polarité)
Divers	—	**redémarrages** (suite de Luby × 100 conflits), **nettoyage** des clauses apprises (critère LBD de Glucose, on conserve les clauses de LBD ≤ 2 et les clauses raisons)

Littéraux codés `2v` / `2v+1` (négation = `l ^ 1`) pour indexer directement des listes Python, boucles internes avec variables locales : en Python, le coût d'un attribut ou d'un appel de fonction domine.

Portfolio CDCL + DPLL (`solver/portfolio.py`)

La Q5 a montré une famille où le CDCL est battu : le principe des tiroirs (`hole9` : CDCL > 120 s, DPLL_H1 22 s), parce que toutes les branches se valent et que seul compte le coût par nœud. Plutôt que de deviner la famille d'une instance, le mode `optimized` combine les deux solveurs :

1. le CDCL travaille **seul pendant 2 s** — 897 de nos 900 instances sont résolues dans ce délai, sans aucun surcoût ;
2. sinon, un **processus fils** exécute DPLL_H avec H1 (coût par décision le plus faible) pendant que le CDCL poursuit dans le processus principal — la compétition accorde 4 CPU par exécution, le second processus ne ralentit donc pas le premier ;
3. la première réponse est retenue et l'autre processus est arrêté. Un modèle venant du fils est revérifié contre toutes les clauses avant d'être affiché ;
4. le fils surveille le processus principal et s'arrête dès qu'il disparaît : si l'infrastructure tue `sat.py` à la limite de temps, aucun processus ne lui survit (voir les résultats ci-dessous).

Pour pouvoir consulter régulièrement le processus fils, le CDCL a été rendu **interruptible et reprenable** : `solve(timeout)` lève `TimeoutError` uniquement juste après une décision (la propagation précédente s'est terminée sans conflit : l'état est celui du début d'une itération) ; un nouvel appel reprend la recherche telle

quelle, sans perdre la pile, les clauses apprises, les activités ni la suite de Luby. Un test unitaire interrompt la recherche à chaque décision et vérifie que statut et modèle restent corrects. Processus créé par `fork` sous Linux (démarrage immédiat, la formule déjà lue est partagée), par `spawn` sous Windows.

Résultats (simulation de la compétition)

`experiments/competition.py` reproduit la procédure officielle : `python sat.py instance.cnf --output-model` en sous-processus, limite de 60 s, contrôle du format exact de la sortie, puis du statut (comparé au statut attendu : `expected.json` pour les instances publiques, `pycosat` ou nom SATLIB pour les autres) et du modèle (vérifié contre toutes les clauses). Comme dans le règlement, une instance n'est comptée que si statut et modèle sont corrects.

Mesures dans l'environnement de la compétition (Docker `python:3.12-slim`, `--cpus=4 --memory=8g`, limite 60 s) sur **900 instances** : les 41 instances publiques et nos 859 instances (`data/examples`, `data/satlib`, `data/generated` : exemples de l'énoncé, SATLIB, 3-SAT généré, PHP, grilles Meowdoku). Mode `optimized` exécuté une instance à la fois, mode `dpll` quatre à la fois (`results/competition/`) :

mode	résolues	dont publiques	délais dépassés	réponses fausses	temps total (résolues)	médiane	pire temps
`dpll` (algorithme 2)	799	38 / 41	101	0	2 200 s	0,61 s	59,4 s
`optimized` (portfolio)	**899**	**41 / 41**	1	0	**248 s**	0,23 s	22,0 s

- **Instances publiques** : le mode `optimized` les résout toutes en 9,7 s au total (pire : 0,39 s), y compris les deux annoncées comme « délibérément difficiles ». Le mode `dpll` dépasse le délai sur `public-03` (= `dubois21`), `public-12` et `public-27`.
- Le mode `dpll` échoue aussi sur `aim-100/200` (40), `uuf100` (26), `uf100` (6), `dubois` (13), 11 grilles Meowdoku de 12 × 12 à 24 × 24 et `hole9`/`hole10` (quatre exécutions simultanées : ses temps sont un peu pessimistes).
- Le mode `optimized` résout tout sauf `hole10`. Son temps médian (0,23 s) est essentiellement le **démarrage de l'interpréteur Python** : une formule de 1 à 4 clauses (`lampes`, `smoke_01`, `unit_01`) prend 0,19 à 0,23 s. 885 des 899 instances résolues le sont en moins de 0,5 s, et seules 2 dépassent les 2 s au bout desquelles le DPLL est lancé (`hole8` : 5,2 s, `hole9` : 22,0 s). Le portfolio n'a donc de coût que là où il sert : `hole9`, hors de portée du CDCL seul (> 120 s), est résolu en 22 s.
- **Processus auxiliaire et limite de temps.** Une première campagne avait mesuré 48 s pour `hole9` : `hole10`, exécuté juste avant (ordre alphabétique), était tué à 60 s par `subprocess.run`, mais son DPLL auxiliaire lui survivait, orphelin, et occupait un CPU pendant les instances suivantes. L'infrastructure officielle procède de même : le processus fils surveille désormais le processus principal et s'arrête dès qu'il disparaît (`portfolio._watch_parent`, test `test_portfolio_helper_dies_with_parent`).
- Les deux modes renvoient toujours le même statut, conformément à l'énoncé.

Éléments originaux, limites et pistes

- Implémentation de **toutes** les heuristiques demandées (statiques et dynamiques) et de deux heuristiques personnelles, dont H9, conçue à partir de l'analyse des mesures et très efficace sur le Meowdoku.
- Générateur avec option d'**unicité** de la solution par réparations locales.
- Vérification systématique : tous les modèles produits sont vérifiés, tous les solveurs sont comparés entre eux, à `pycosat` et à une énumération exhaustive ; la solution d'une grille est vérifiée par un contrôleur indépendant de l'encodage.

- Simulation de la compétition (`experiments/competition.py`) appelant `sat.py` en sous-processus, vérifiant le format exact de la sortie, le statut attendu (`expected.json` des instances publiques) et le modèle, exécutée dans l'image Docker et avec les limites de la compétition.
- Portfolio CDCL + DPLL (Q6) conçu à partir d'une mesure de la Q5 (principe des tiroirs), avec un CDCL interruptible et reprenable et un processus auxiliaire qui ne survit pas à la limite de temps.
- Toutes les mesures du rapport sont reproductibles par les scripts de `experiments/` et conservées dans `results/` ; les instances générées sont reproduites à l'identique par `experiments/gen_cnf.py` (graines fixes).
- **Limites.** Le solveur reste en Python pur : sur 39 instances (`uuf75`, `uf100`, `flat50`, grilles 20 × 20 et 24 × 24, 3 instances publiques), notre CDCL est 6 fois plus lent que PicoSAT, écrit en C (moyenne géométrique ; de 2 à 43 fois ; temps de PicoSAT mesuré via `pycosat`, conversion des clauses comprise, ce qui avantage notre solveur). Les instances intrinsèquement dures pour la résolution restent hors de portée au-delà d'une certaine taille (`hole10` : ni le CDCL ni le DPLL ne concluent en 60 s) ; `hole9` est résolu en 22 s, à un peu plus d'un tiers de la limite. L'option d'unicité du générateur devient lente au-delà de 12 × 12 (1 à 10 s jusqu'à 12 × 12, plus de 5 min en 14 × 14).
- **Pistes.** Pré-traitement (élimination de variables, subsomption), littéraux bloqueurs dans les listes de surveillance, minimisation récursive des clauses apprises, redémarrages adaptatifs (Glucose), détection et codage spécifique des contraintes de cardinalité.

Références

- M. Davis, H. Putnam. *A computing procedure for quantification theory*. JACM 7(3), 1960.
- M. Davis, G. Logemann, D. Loveland. *A machine program for theorem-proving*. CACM 5, 1962.
- M. Lagoudakis, M. Littman. *Learning to select branching rules in the DPLL procedure for satisfiability*. ENDM 9, 2001.
- R. Jeroslow, J. Wang. *Solving propositional satisfiability problems*. Annals of Math. and AI 1, 1990.
- J. W. Freeman. *Improvements to propositional satisfiability search algorithms* (POSIT). PhD thesis, 1995.
- J. Marques-Silva, K. Sakallah. *GRASP: a search algorithm for propositional satisfiability*. IEEE ToC, 1999.
- M. Moskewicz et al. *Chaff: engineering an efficient SAT solver*. DAC 2001.
- N. Eén, N. Sörensson. *An extensible SAT-solver* (MiniSat). SAT 2003.
- G. Audemard, L. Simon. *Predicting learnt clauses quality in modern SAT solvers* (Glucose). IJCAI 2009.
- C. Sinz. *Towards an optimal CNF encoding of Boolean cardinality constraints*. CP 2005.
- H. Hoos, T. Stützle. *SATLIB: an online resource for research on SAT*. SAT 2000.